

Úvod do objektově orientovaného programování

Základy programování

Matouš Cejnek
U12110, FS, ČVUT v Praze

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Co si dnes odnesete?

- Rozlišit třídu, objekt a instanci.
- Vytvořit třídu s atributy, konstruktorem a metodami.
- Vysvětlit roli `self` a sledovat změny stavu objektu.
- Rozlišit sdílený odkaz, mělkou kopii a hlubokou kopii.
- Číst jednoduchý diagram tříd a sekvenční diagram.
- Použít dědičnost, přepsání metody a polymorfismus.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Na co navazujeme?

- Funkce pojmenovávají chování a přijímají parametry.
- Seznamy a slovníky seskupují hodnoty.
- Objekt spojí **data a operace nad nimi** do jednoho celku.

OOP je nástroj organizace

Ne každý program potřebuje vlastní třídy. Používáme je, když model zpřehlední.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Samostatná data se mohou rozcházet

```
names = ["Eva", "Petr"]
balances = [700, 1200]

balances[0] += 200
```

- Vazbu mezi jménem a zůstatkem drží jen stejný index.
- Operace nad účtem jsou rozptýlené po programu.
- Chceme mít data i povolené změny pohromadě.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Třída a objekt

Třída předpis společné struktury a chování, například `BankAccount`.

Objekt konkrétní hodnota vytvořená podle třídy, například účet Evy.

Instance jiné označení objektu dané třídy.

Jedna třída, mnoho objektů

Každý objekt má vlastní stav, ale sdílí definici metod své třídy.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

První třída

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
```

Název třídy se podle konvence píše `CamelCase`; názvy metod jako funkce.

Vytvoření objektu a volání metody

```
eva = BankAccount("Eva", 700)
petr = BankAccount("Petr")

eva.deposit(200)
print(eva.balance)    # 900
print(petr.balance)   # 0
```

- Volání třídy vytvoří nový objekt.
- Tečka zpřístupní atribut nebo metodu konkrétního objektu.

Atribut, metoda, stav

Atribut pojmenovaná hodnota patřící objektu: `eva.balance`.

Metoda funkce definovaná ve třídě: `eva.deposit(200)`.

Stav aktuální soubor hodnot atributů objektu.

Metoda může stav číst, změnit nebo z něj vypočítat výsledek.

Co dělá `__init__`?

```
eva = BankAccount("Eva", 700)
```

1. Python vytvoří nový prázdný objekt.
2. Automaticky zavolá `__init__(nový_objekt, "Eva", 700)`.
3. `__init__` nastaví počáteční stav objektu.

Přesný pojem

`__init__` je inicializátor; v úvodním kurzu se často zjednodušeně nazývá konstruktor.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Co znamená `self`?

`self`

Odkaz na konkrétní objekt, se kterým právě probíhá volání metody.

- `eva.deposit(200)` odpovídá `BankAccount.deposit(eva, 200)`.
- Python první argument doplní při volání přes objekt.
- Jméno `self` je konvence, kterou vždy dodržujeme.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Parametr a atribut nejsou totéž

```
def __init__(self, owner, balance=0):
    self.owner = owner
    self.balance = balance
```

- `owner` je lokální parametr tohoto volání.
- `self.owner` je atribut, který zůstane v objektu.
- Pravá strana se vyhodnotí a levá určuje, kam hodnotu uložit.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Každý objekt má vlastní stav

```
a = BankAccount("Eva", 700)
b = BankAccount("Petr", 700)
a.deposit(200)

print(a.balance, b.balance)  # 900 700
```

Proměnné a a b odkazují na dva různé objekty.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Metoda může vracet hodnotu

```
class BankAccount:
    # __init__ and deposit are unchanged

    def can_withdraw(self, amount):
        return amount <= self.balance

    def describe(self):
        return f"{self.owner}: {self.balance:.2f} CZK"
```

Stejně jako u funkcí rozlišujeme výpis pomocí `print` a návrat hodnoty.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Metoda může hlídat platné změny

```
def withdraw(self, amount):
    if amount <= 0:
        return False
    if amount > self.balance:
        return False
    self.balance -= amount
    return True
```

Zapouzdření

Objekt nabízí operace, které chrání pravidla jeho stavu; uživatel nemusí znát vnitřní postup.

Metoda může volat jinou metodu

```
def transfer_to(self, other, amount):
    if not self.withdraw(amount):
        return False
    other.deposit(amount)
    return True

eva.transfer_to(petr, 300)
```

`self` je odesílající účet; `other` odkazuje na jiný objekt.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Objekt je také hodnota

- Lze ho uložit do proměnné nebo seznamu.
- Lze ho předat jako argument funkci či metodě.
- Více proměnných může odkazovat na tentýž objekt.

Důsledek sdíleného odkazu

Změna objektu přes jeden odkaz je vidět i přes druhý odkaz na stejný objekt.

Opakování: přiřazení a neměnné hodnoty

```
a = 10
b = a
b += 5
print(a, b)                    # 10 15
```

- Proměnná je jméno odkazující na objekt; `b = a` objekt nekopíruje.
- `int`, `float`, `bool` a `str` jsou **neměnné** typy.
- `b += 5` zde naváže jméno `b` na výsledek výpočtu; hodnotu `a` nezmění.

Chování připomíná kopii hodnoty, ale pravidlo přiřazení je pro všechny typy stejné.

Opakování: seznam a objekt lze měnit

```
items = [10]
alias = items
alias.append(20)
print(items)                # [10, 20]
a = BankAccount("Eva", 700)
b = a
b.deposit(100)
print(a.balance, a is b)    # 800 True
```

`list`, `dict` i naše účty jsou **měnitelné**. `is` porovnává totožnost objektů; `==` porovnává jejich hodnoty.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Mělká kopie (shallow copy)

```
original = [[1, 2], [3, 4]]
shallow = original.copy()
shallow.append([5, 6])
print(len(original))        # 2
shallow[0].append(99)
print(original[0])          # [1, 2, 99]
```

- Vznikne nový vnější seznam, ale odkazy na jeho prvky se sdílejí.
- Vnořený seznam proto změnou zasáhneme i přes kopii.
- Pro seznam má stejný účinek `original[:]` nebo `list(original)`.

Hluboká kopie (deep copy)

```
from copy import deepcopy
original = [[1, 2], [3, 4]]
deep = deepcopy(original)
deep[0].append(99)
print(original[0])          # [1, 2]
print(deep[0])              # [1, 2, 99]
print(deep[0] is original[0])  # False
```

- Rekurzivně kopíruje i vnořené měnitelné objekty.
- Neměnné hodnoty mohou zůstat sdílené; jejich obsah nelze změnit.
- Použijeme ji, když potřebujeme nezávisle měnit i vnitřní data.

Kopírování vlastních objektů

```
from copy import copy, deepcopy
accounts = [BankAccount("Eva", 700)]
shallow = copy(accounts)
deep = deepcopy(accounts)
shallow[0].deposit(100)
deep[0].deposit(50)
print(accounts[0].balance, deep[0].balance)  # 800 750
```

- `copy(obj)` vytvoří mělkou kopii i běžné vlastní instance: atributy sdílí.
- `deepcopy(obj)` kopíruje i objekty v jejích atributech.
- Kopírujeme data, která mají být nezávislá; sdílené účty obvykle chceme sdílet dál.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

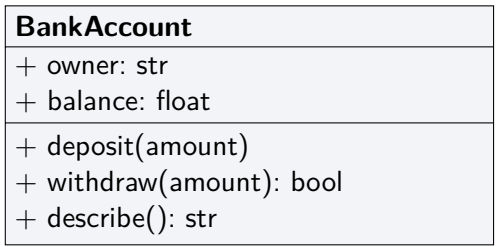
Kdy třída pomáhá?

```
accounts = [
    BankAccount("Eva", 700),
    BankAccount("Petr", 1200),
    BankAccount("Jana", 300),
]

for account in accounts:
    print(account.describe())
```

Třída dává všem prvkům stejné rozhraní a každému vlastní stav.

UML diagram třídy



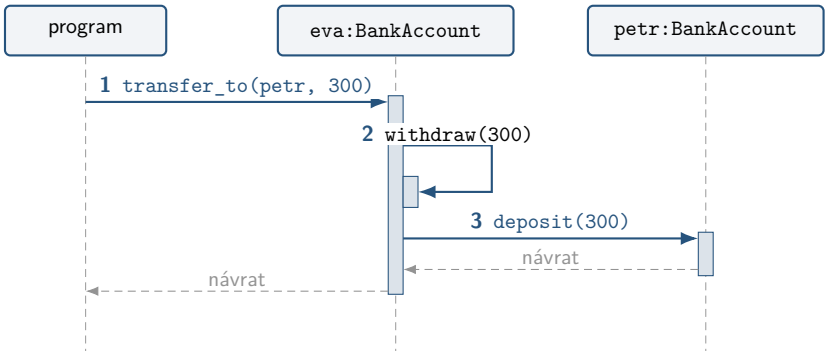
- Horní část: jméno třídy; prostřední: atributy; dolní: metody.
- + značí veřejnou část rozhraní, − neveřejný detail.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

UML není kód

- Diagram ukazuje strukturu a vztahy, ne přesný postup metod.
- Typ za dvojtečkou pomáhá čtenáři, i když jej Python nemusí vynucovat.
- Závorky rozlišují metodu od atributu.
- Diagram třídy popisuje předpis, nikoli aktuální hodnoty jednoho objektu.

Sekvenční diagram: pořadí volání



Čas plyne shora dolů; smyčka značí, že objekt volá vlastní metodu.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Diagram tříd vs. sekvenční diagram

Diagram tříd

Statická struktura: třídy, atributy, metody a vztahy.

Sekvenční diagram

Dynamický průběh: konkrétní objekty a pořadí volání v čase.

Dědičnost: specializace existující třídy

Dědičnost

Potomek přebírá chování rodiče a může ho rozšířit nebo upravit.

- `Employee` je obecný zaměstnanec.
- `Doctor` je specializovaný zaměstnanec.
- Dědičnost vyjadřuje vztah „je druhem“ (*is-a*).

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Základní zápis dědičnosti

```
class Employee:
    def describe(self):
        return "employee"

class Doctor(Employee):
    def examine(self, patient):
        return f"Examining {patient}"

d = Doctor()
print(d.describe())       # inherited method
```

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Přepsání metody (override)

```
class Employee:
    def describe(self):
        return "employee"

class Doctor(Employee):
    def describe(self):
        return "doctor"

print(Doctor().describe())  # doctor
```

Potomek definuje metodu stejného jména; při volání se použije jeho verze.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

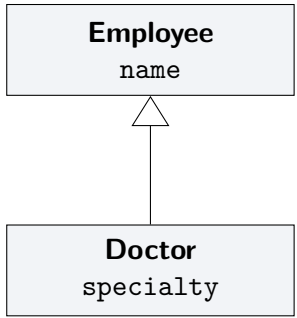
Vlastní `__init__` přepíše rodičovský

```
class Employee:
    def __init__(self, name):
        self.name = name

class Doctor(Employee):
    def __init__(self, name, specialty):
        super().__init__(name)
        self.specialty = specialty
```

- `super()` zpřístupní implementaci rodiče.
- Bez volání rodiče by atribut `name` nevznikl.

Dědičnost v UML



- Dutý trojúhelník míří od potomka k rodiči.
- Potomek dědí členy rodiče a přidává vlastní.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Polymorfismus: stejné volání, různé chování

```
class CashPayment:
    def process(self):
        return "Paid in cash"

class CardPayment:
    def process(self):
        return "Paid by card"

for payment in [CashPayment(), CardPayment()]:
    print(payment.process())
```

Kód používá společné rozhraní a nemusí větvit podle konkrétního typu.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Kompozice: objekt obsahuje jiné objekty

- `Warehouse` uchovává seznam objektů `Item`.
- `Appointment` odkazuje na objekt `Patient` i `Doctor`.
- Kompozice vyjadřuje vztah „má“ (*has-a*).

Praktické pravidlo

Pro vztah „má“ preferujte kompozici; dědičnost použijte pro skutečnou specializaci „je druhem“.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Speciální metody zapojují objekty do Pythonu

```
class Vector2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

print(Vector2D(3, 4))    # (3, 4)
```

Podobně `__add__` určuje chování operátoru + a `__eq__` porovnání ==.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Kdy třídu raději nevytvářet?

- Jednorázový výpočet bez dlouhodobého stavu často stačí jako funkce.
- Třída pouze s jednou metodou bez atributů může být zbytečná vrstva.
- Nejprve se ptejte: existuje zde několik hodnot stejného druhu s vlastním stavem a chováním?

Cíl není „použít OOP“

Cílem je kód, jehož struktura odpovídá problému a dobře se mění.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Časté chyby

- Chybí dvojtečka za `class` nebo `def`.
- Metodě chybí první parametr `self`.
- Místo `self.name` se použije neexistující lokální `name`.
- Konstruktor se nejmenuje přesně `__init__`.
- Metoda je jen pojmenována: `account.deposit` místo volání se závorkami.
- Potomek přepíše `__init__`, ale neinicializuje rodičovské atributy.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Rychlá kontrola porozumění

1. Jaký je rozdíl mezi třídou a objektem?
2. Na co odkazuje `self` při `a.deposit(100)`?
3. Proč mají dva účty rozdílný zůstatek?
4. Kdy nestačí mělká kopie seznamu objektů?
5. Co ukazuje diagram tříd a co sekvenční diagram?
6. Co se stane, když potomek definuje metodu stejného jména jako rodič?

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Co využijete na cvičení?

- `Rectangle`: atributy, metody a změna stavu.
- `Vector2D`: metody vracející nové objekty a `__str__`.
- `Warehouse` a `Item`: spolupráce objektů a kompozice.
- Hierarchie tvarů: dědičnost, přepis metod a polymorfismus.
- Pokročilé úlohy: další speciální metody a návrh více tříd.

ČVUT
ČESKÉ VYSOKÉ UČENÍ TECHNICKÉ V PRAZE

Shrnutí

- Třída je předpis, objekt je konkrétní instance s vlastním stavem.
- Atributy uchovávají data, metody definují chování.
- `self` označuje objekt aktuálního volání; `__init__` nastaví jeho počáteční stav.
- UML diagram tříd popisuje strukturu, sekvenční diagram průběh volání.
- Dědičnost specializuje, kompozice skládá a polymorfismus sjednocuje rozhraní.